Lab 01 — Commented answers

Each answer follows the same pattern: the answer, the mechanism, the nuance or pitfall, and an example you can verify at the terminal.

Question 1 — "A container is a small VM"

Answer. Wrong on the essential point: a container contains **no operating system** and has **no kernel of its own**. It is a host process, isolated by *namespaces*, limited by *cgroups*, and run by the host kernel.

Why. A VM boots a kernel, then an `init`, then dozens of system services (logging, cron, SSH, networking…) before your application even starts. That is where the seconds or minutes of boot time and the gigabytes of disk go. A container boots none of that: the kernel is already running, and we merely ask it to create namespaces and launch **one** process. Starting one costs a `fork` + `exec` — a few milliseconds; on disk it costs only the libraries the application actually needs.

> **→ LINUX**
> `fork` duplicates the current process; `exec` replaces its content with another program. Every Linux process is born this way, `podman run` included: Podman duplicates itself, the clone enters its namespaces, then executes your command. A container comes into the world exactly like an `ls`.

Nuance. For a *user*, the "light VM" intuition is not unreasonable: you do get a `/`, a `hostname`, an IP, and a `root`. It turns dangerous the moment security enters the picture. A VM's hypervisor is a real boundary; a container shares the kernel, and one kernel flaw walks straight through it. On Windows, the "lightness" is also relative: there is a VM after all — WSL 2 — but a single one shared by all your containers.

Example.

```
time podman run --rm alpine echo hello    # ~0.3 s, mostly the pull/CLI
podman run -d nginx:alpine                # ~64 MB image, PID visible on the host via ps
```

Question 2 — 250 MB "without an OS"

Answer. The image contains a distribution's **userland**: `/bin/sh`, `libc`, `coreutils`, the PostgreSQL binaries, the configuration. What is **missing** is the **kernel** — and with it everything that exists only at boot time: bootloader, `initrd`, kernel modules, `systemd`, drivers, hardware management.

Why. A Linux program talks to the kernel through system calls (`open`, `read`, `fork`). It does not need to ship a kernel; it only needs to find one, and the host's will do. The image supplies only what is missing above that line.

Nuance. This is why an "Alpine" image and a "Debian" image run side by side on the same Ubuntu under WSL: three userlands, one kernel. It is also why a Linux image cannot run on a Windows kernel — hence WSL.

Example.

```
podman run --rm alpine cat /etc/os-release | head -1   # NAME="Alpine Linux"
uname -r                                               # 6.6.87.2-microsoft-standard-WSL2
podman run --rm alpine uname -r                        # STRICTLY the same kernel
```

Question 3 — Two nginx containers, two different `ps`

Answer. The **`pid` namespace**. Each container gets its own PID table: its main process is numbered 1 there, and no outside PID is visible.

Why. Seeing a process is a prerequisite for acting on it (`kill`, `/proc/<pid>`). By removing visibility, the kernel effectively removes the ability to do harm through that route. On the host, those processes exist just fine, with real PIDs.

Nuance. This is not security "in the strict sense" because the boundary is **not impassable**: it is a visibility restriction, enforced by the very kernel the container runs on. Start a container with `--pid=host` or `--privileged` and the full view comes back; a kernel flaw bypasses the mechanism entirely. The namespace isolates; it does not defend. In rootless mode, the `user` namespace does add a real barrier of *rights* on top of this barrier of *visibility*.

Example.

```
podman run -d --name web nginx:alpine
podman exec web ps -o pid,comm            # PID 1 = nginx, then its workers
ps -ef | grep -c "[n]ginx"                # on the host: the processes are there
podman run --rm --pid=host alpine ps | head   # isolation removed: the whole WSL
podman rm -f -t 0 web
```

Question 4 — `Cannot connect to the Docker daemon`

Answer. On your colleague's machine, the client did its job: it printed its version, then tried to open the `/var/run/docker.sock` socket to reach the **server**, and failed there. Two likely causes: (1) the daemon is not running, (2) their user has no permission on the socket. On your machine this message cannot appear: Podman has **no daemon** and opens no socket; each command does the work itself, under your user.

Why. Every Docker command is, at heart, a network call to `dockerd`. Tweaking the command changes nothing, because nothing has read it yet: the problem sits one step earlier. Podman is an ordinary program: if it starts, it works.

Nuance. Podman *does* have a server in two cases: `podman --remote` (or the `CONTAINER_HOST` variable), which talks to a remote `podman system service`, and `podman machine` on Windows/macOS, where the Windows client talks to a VM. There you would see `unable to connect to Podman socket`. With Podman installed directly in Ubuntu under WSL, neither case applies to you.

Example.

```
# Docker side, the diagnosis:
systemctl status docker             # cause 1: inactive (dead)
ls -l /var/run/docker.sock          # srw-rw---- root docker: you must be in the docker group
# Podman side, proof that there is nothing to contact:
podman version                      # a single "Client" block
podman --remote version             # Error: unable to connect to Podman socket …
```

Question 5 — "An image does not run"

Answer. An image is a set of read-only files plus metadata. There is no process, no state, nothing to schedule: it is as inert as a `.zip` with an instruction sheet.

Why. At `podman run`, the engine adds three things: (1) a thin **writable layer** on top of the read-only layers, (2) a set of **namespaces and cgroups**, (3) the **execution** of the command recorded in the image metadata (`ENTRYPOINT` / `CMD`), through the `crun` runtime. The result of that assembly is the container.

Nuance. Creating a container and starting it are two distinct steps: `podman create` does everything except launch the process; `podman start` launches it. `podman run` is simply `create` + `start` (+ `pull` if the image is missing).

Example.

```
podman create --name tmp alpine sleep 30   # container created, no process
podman ps -a --filter name=tmp             # STATUS: Created
podman start tmp && podman ps              # STATUS: Up -> now it runs
podman rm -f -t 0 tmp
```

Question 6 — PostgreSQL data after a `podman rm`

Answer. No, the data is gone. And no, the image was **not** modified: it is strictly identical before and after.

Why. Everything a container writes lands (via *copy-on-write*) in its private writable layer. `podman rm` removes the container **and** that layer. The image layers are read-only: nothing a container does can touch them — which is exactly what guarantees that two containers from the same image start from the same state.

Nuance. Two important corrections. First, `podman stop` destroys nothing: a stopped container keeps its layer, and `podman start` brings the data back. It is `rm` that destroys. Second, the official `postgres` image declares `/var/lib/postgresql/data` as a `VOLUME`, so the engine creates an **anonymous** volume that survives the `rm` — but with no name, you will almost never find it again. In practice, treat the data as lost. Named volumes are the subject of lab 06.

Example.

```
podman run -d --name c1 alpine sleep 600
podman exec c1 sh -c 'echo x > /mark.txt'
podman rm -f -t 0 c1
podman run --rm alpine ls /mark.txt      # No such file or directory: the image is intact
```

Question 7 — Ten containers, how much disk?

Answer. A few dozen kilobytes in total — not 10 × 210 MB. Each container costs only its writable layer, which starts out empty, plus a handful of configuration files (`hostname`, `resolv.conf`...).

Why. All containers created from an image **share** its layers read-only. The `overlay` storage driver stacks those layers with one empty writable layer per container on top. A file is copied into that layer only at the moment it is modified (*copy-on-write*).

Nuance. The answer changes if each container writes heavily (logs, temporary files): modifying any file from the image copies it wholesale into the container's layer. `podman ps -s` shows both figures: the container's own size and the "virtual" size.

Example.

```
for i in 1 2 3; do podman run -d --name t$i alpine sleep 600; done
podman ps -s --format 'table {{.Names}}\t{{.Size}}'   # 11.4kB (virtual 8.72MB) each
podman rm -f -t 0 t1 t2 t3
```

Question 8 — `root` inside, `1000` on the host

Answer. The `user` namespace maps the container's identifiers onto the host's: the container's UID 0 *is* your UID 1000. Toward the kernel and the host's files, that "root" holds nothing more than your rights. An attempt to write to `/etc/shadow` mounted from the host fails with `Permission denied`, exactly as if you had tried it yourself.

Why. The kernel checks permissions against the **real** identity (the host-side one), not the identity displayed inside the namespace. The container's UIDs 1 through 65536 are mapped onto a reserved range in `/etc/subuid` (`100000-165535`) that has no rights on your files.

Nuance. Inside its own namespaces, that root *is* root: it can install packages, change permissions on the image's files, and listen on the container's port 80. What it cannot do is cross the boundary. A practical consequence: a file the container creates under UID 999 (the `postgres` user) shows up on your host with UID 100998 — the classic *bind mount* trap in rootless mode (lab 06).

Example.

```
podman top watcher user,huser               # root / 1000
podman unshare cat /proc/self/uid_map       # 0 -> 1000 (1), 1 -> 100000 (65536)
podman run --rm -v /etc:/host alpine sh -c 'echo x >> /host/shadow'   # Permission denied
```

Question 9 — The `docker` group and `sudo`

Answer. Membership in the `docker` group grants write access to `/var/run/docker.sock` — and therefore the power to have anything executed by a daemon running as **root**: `docker run -v /:/host --privileged` hands over the entire host. It amounts to `sudo` with no password, no logging, and no limits. With rootless Podman there is no root daemon and no socket: the user can do nothing they could not already do, so the rule no longer has anything to protect against.

Why. Auditing requires knowing *who* did *what*. A command sent through the socket is executed by `dockerd`, under the `root` identity, with no trace tied to the user. `sudo docker …` at least leaves a line in `auth.log`. Rootless Podman goes further: the container is a process owned by the user, visible and attributable in `ps`.

Nuance. Rootless Podman has its costs: no ports below 1024 without tuning, a slightly slower user-space network, and some mounts and options are off-limits. In production you will also meet *rootful* Podman (`sudo podman`) — which brings back the same precautions as Docker.

Example.

```
# What the docker group allows (DO NOT do this on a shared machine):
docker run --rm -v /:/host alpine cat /host/etc/shadow     # readable: the daemon is root
# The same thing under rootless Podman:
podman run --rm -v /:/host alpine cat /host/etc/shadow     # Permission denied
```

Question 10 — Long form, short form

Answer.

Shortcut	Long form	Object
`podman ps -a`	`podman container ls -a`	container
`podman images`	`podman image ls`	image
`podman rmi nginx:alpine`	`podman image rm nginx:alpine`	image
`podman rm web`	`podman container rm web`	container

Why. `ps` and `images` date back to the earliest Docker releases (2013), when the CLI had no notion of objects: `ps` imitated the Unix command of the same name, and `images` was just a plural. The `object action` grammar arrived in 2017 (Docker 1.13), and Podman adopted it unchanged. The shortcuts survive so that nothing breaks.

Nuance. Only the long form is complete: `podman container ls`, `podman image ls`, `podman volume ls`, `podman network ls`, and `podman pod ls` all follow the same pattern, whereas `podman ps` has no counterpart for volumes. In scripts, prefer the long form.

Example.

```
podman container ls -a --format '{{.Names}}'
podman image ls --format '{{.Repository}}:{{.Tag}}'
```

Question 11 — Two `uname -r`, two hosts

Answer. `uname -r` is a system call: the value comes from the **kernel**, never from the image. Under WSL, that kernel is `microsoft-standard-WSL2`, which Microsoft compiles for the WSL VM; on a native Ubuntu server, it is the `generic` kernel from the Ubuntu package. A container always reports the kernel of the machine running it, whatever the image.

Why. The container is a process of the host kernel, and the image contains no kernel (question 2). On Windows, that host is not Windows itself but the WSL 2 VM.

Nuance. "Lightweight" still holds: there is only **one** WSL VM, started once and shared by all your containers, and the containers themselves remain processes that start in milliseconds. What no longer holds is "no VM at all". Practical consequences: the available RAM is WSL's (`.wslconfig`), and Windows files (`/mnt/c/…`) mounted into a container are slow, because every access crosses the VM ↔ Windows boundary. Work inside the Linux file system (`~`).

Example.

```
uname -r                            # 6.6.87.2-microsoft-standard-WSL2
podman run --rm alpine uname -r     # identical
podman info --format '{{.Host.Kernel}} {{.Host.MemTotal}}'   # the RAM as seen by WSL
```

Question 12 — Infinite loop and RAM

Answer. No: the `pid` namespace only hides processes. The mechanism that protects the neighbors is the memory **cgroup**, enabled with `--memory`. Without a limit, the container consumes all available RAM, and once the kernel has nothing left, the **OOM killer** kills a process of its own choosing — not necessarily the culprit.

Why. Namespaces and cgroups are two independent mechanisms: one isolates the *view*, the other caps *consumption*. With `--memory=512m`, exceeding the limit kills only the container's own process (`Exited (137)`, `OOMKilled: true`), and the rest of the machine never notices.

Nuance. In rootless mode, `--memory` works only if `systemd` delegates the `memory` controller to your user — which Ubuntu under WSL does once `systemd=true` is enabled. And for Java, a cgroup limit helps only if the JVM honors it: since Java 10 it reads the cgroup automatically (`-XX:MaxRAMPercentage`), but an `-Xmx` set too high by hand will blow past it anyway.

Example.

```
podman run -d --name limited --memory=128m --memory-swap=128m alpine sleep 600
podman stats --no-stream limited     # MEM USAGE / LIMIT: … / 134.2MB
podman inspect --format '{{.State.OOMKilled}}' limited   # false — for now
podman rm -f -t 0 limited
```

Question 13 — Promoting an image without rebuilding it

Answer. Two properties: **immutability** (an image identified by its digest never changes) and the **OCI standard** (Docker and Podman write and read exactly the same format). What was tested in staging goes to production bit for bit identical, whatever the engine. Rebuilding at every stage breaks that guarantee: two builds of the same code do not necessarily produce the same image.

Why. A build depends on when it runs: `apt-get install` picks up that day's version, `FROM eclipse-temurin:21-jre` follows a moving tag, Maven resolves version ranges. Between the staging build and the production build, a dependency may have changed — and the staging validation is now worthless.

Nuance. Promotion does not mean re-tagging `latest`; it means referencing the **digest** (`api@sha256:…`) or an immutable tag (`api:1.4.2`). Lab 02 covers this in depth. And the engine hardly matters: pulling with `podman pull` an image pushed with `docker push` is entirely routine.

Example.

```
podman image inspect --format '{{.Digest}}' registry.internal/api:1.4.2
# Same digest on the workstation (Podman), in staging (Docker) and in production (rootful Podman).
```

Question 14 — `alias docker=podman`

Answer. True without reservation for: (1) the whole image cycle — `build`, `pull`, `push`, `tag`, `images`, `history`, `inspect`, Dockerfiles; (2) the container life cycle — `run`, `ps`, `logs`, `exec`, `stop`, `rm`, options included. False or different: (a) **no daemon** — no `docker.sock`, `--restart=always` does not survive a reboot without `systemd`, `podman rm -f` waits 10 s; (b)

rootless — no ports below 1024 without tuning, no IP addresses with `pasta`, shifted UIDs on *bind mount* files, `--memory` dependent on cgroup delegation.

Why. Podman copied Docker's *surface* (the CLI, the format) but not its *architecture*. Everything that depends only on the surface is identical; everything that touches "who executes, with what rights, supervised by whom" diverges.

Nuance. These differences are not defects: each one is the flip side of a security choice. And Docker Compose does work with Podman (`podman compose`, lab 09), at the cost of a few lines of configuration.

Example.

```
alias docker=podman
docker run -d --name web -p 8080:80 nginx:alpine   # identical
docker run -d --name w80 -p 80:80 nginx:alpine     # Error: pasta failed … Listen failed for
HOST TCP port */80: Permission denied
podman rm -f -t 0 web
```